Edge Control User Manual

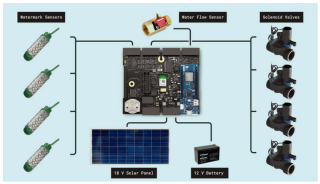
Learn about the hardware and software features of the Arduino® Edge Control.

Author · Christopher Méndez Last revision · 09/23/2024

ON THIS PAGE

Overview

This user manual will guide you through a practical journey covering the most interesting features of the Arduino Edge Control. With this user manual, you will learn how to set up, configure and use this Arduino board.



Agriculture Application Example Setup

Hardware and Software Requirements

Hardware Requirements

- Arduino Edge Control (x1)
- Micro USB cable (x1)
- External power source: a 12V SLA battery or 12V power supply (x1)

Software Requirements

- Arduino IDE 1.8.10+, Arduino IDE 2.0+, or Arduino Web Editor

Product Overview

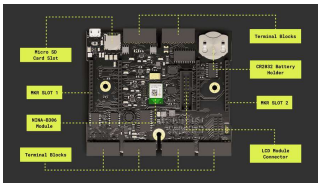
develop creative and innovative solutions for agriculture by harnessing modern technology.

This board is designed to address the needs of precision farming. It provides a low power control system, suitable for irrigation with modular connectivity. Equipped with latching outputs and solid state relays, it becomes an ideal choice for controlling motorized or solenoid valves, among many other devices.

As inputs, it manages 0-5V analog sensors, 4-20mA sensors, and Watermark soil moisture sensors, a variety that handles most of the agricultural needs. All this and more makes the Edge Control the perfect option for your 4.0 Agriculture industry.

Board Architecture Overview

The Edge Control features a robust and efficient architecture integrating a wide range of sensor inputs and ready-to-use control outputs for industrial farming field devices.



Edge Control main components

Here is an overview of the board's architecture's main components shown in the image above:

Microcontroller: at the heart of the Edge Control is the

The nRF52840 is built around a 32-bit Arm® Cortex®-M4 processor running at 64 MHz.

MKR slots: the on-board MKR slots 1 & 2 can be used to connect Arduino MKR boards to extend the capabilities such as connectivity through LoRa, Wi-Fi, 2G/3G/CatM1/NBIoT, and Sigfox.

Storage: the board includes both a microSD card socket and an additional 2MB Flash memory for data storage. Both are directly connected to the main processor via a SPI interface.

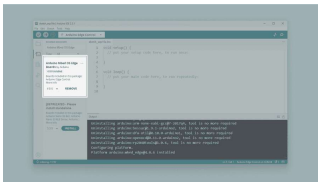
Power management: the Edge Control is designed for ultra-low power operation, with efficient power management features that ensure minimal energy consumption. It can operate for up to 34 months on a 12V/5Ah battery. Equipped with several buck and boost converters supplying a variety of output voltages from 19V DC to 3.3V DC. The LT3652 solar panel battery charger features a Maximum Power Point Tracker (MPPT) taking the best performance out of the solar panels.

Interfaces: through the different terminal blocks, the board give access to several standardized sensors inputs like 0-5V, 4-20mA and watermark sensors. Also, it is equipped with drivered latching outputs, and relay contacts ready to manage high power external devices.

Board Core and Libraries

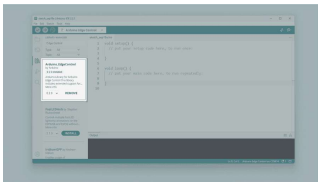
To install the core for the Edge Control, navigate to **Tools > Board > Boards Manager** or click the Boards Manager icon in the left tab of the IDE. Search for `Edge Control` in the

`Arduino Mbed OS Edge Boards` version.



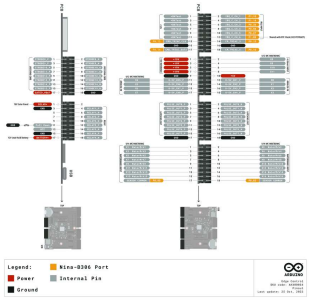
Installing the Arduino Mbed OS Edge Boards core in the Arduino IDE bootloader

The **Arduino_EdgeControl** library contains examples of how to work with the board's components, such as the different sensors and outputs. It adds as the LCD included with the Enclosure Kit.



Installing the Arduino Edge Control library

Pinout



Edge Control pinout

The full pinout is available and downloadable as PDF from the link below:

Datasheet

The complete datasheet is available and downloadable as PDF from the link below:

Edge Control datasheet

Schematics

The complete schematics are available and downloadable as PDF from the link below:

Edge Control schematics

STEP Files

The complete STEP files are available and downloadable from the link below:

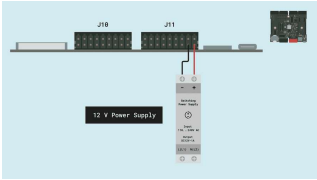
Edge Control STEP files

First Use

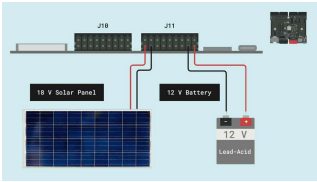
Powering the Board

The Edge Control can be powered by:

- Using a Micro USB cable (not included).
- Using an external **12V power supply** connected to `BATT+` pin and `GND`. (Please refer to the board pinout section of the user manual).
- Using a **12V lead-acid battery** connected to `BATT+` pin and `GND`. *It can be powered for up to 34 months on a 12V/5Ah battery*.
- Using a whole **off grid** power system including an **18V Solar Panel** and a **12V lead-acid battery**.



Edge Control powered by external power supply



Edge Control solar and battery powered

Hello World Example

Let's program the Edge Control with the classic `hello world` example used in the Arduino ecosystem: the `Blink` sketch. We will use this example to verify that the board's connection to the Arduino IDE, the Edge Control core and the board itself are working as expected.

There are two ways to program this example in the board:

Navigate to **File > Examples > Arduino_EdgeControl > Basic > Blink.**

Copy and paste the code below into a new sketch in the Arduino IDE.

```
#include <Arduino_EdgeC

void setup() {
  Serial.begin(9600);

  auto startNow = millis
  while (!Serial && mil
    ;

  delay(1000);
  Serial.println("Hello,

  Power.on(PWR_3V3);
  Power.on(PWR_VBAT);

  Wire.begin();

  delay(500);

  Serial.print("IO Expar
  if (!Expander.begin())
    Serial.println("fail
    Serial.println("Plea
    Serial.println("via
  }
  Serial.println("succe

  Expander.pinMode(EXP_L
```

For the Edge Control, the `EXP_LED1` macro represents the **Green LED** of the board.

The custom power management system of the Edge Control allows you to activate only the specific peripherals and power rails you need. Since the LED is connected to the IO Expander, it is necessary to enable both the 3.3V and the battery power source alongside the IO Expander using the following functions:

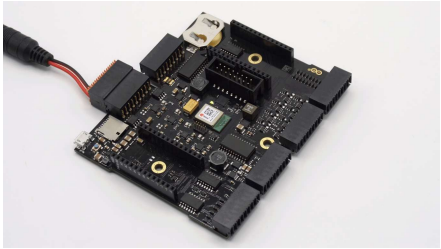
```
Power.on(PWR_3V3);
Power.on(PWR_VBAT);
.
.
```

To upload the code to the Edge Control, click the **Verify** button to compile the sketch and check for errors; then click the **Upload** button to program the board with the sketch.

Uploading a sketch to the Edge Control in the Arduino IDE

The Edge Control should be powered by an external power source or a battery so the blink works.

You should see the onboard LED turn on for half a second, then off, repeatedly.



Inputs

Analog Inputs

The Edge Control has **eight analog input pins**, mapped as follows:

Input Name	Arduino Pin Mapping
0 - 5 V Channel	

Input Name	Arduino Pin Mapping
0 - 5 V Channel 2	INPUT_05V_CH02
0 - 5 V Channel 3	INPUT_05V_CH03
0 - 5 V Channel 4	INPUT_05V_CH04
0 - 5 V Channel 5	INPUT_05V_CH05
0 - 5 V Channel 6	INPUT_05V_CH06
0 - 5 V Channel 7	INPUT_05V_CH07
0 - 5 V Channel 8	INPUT_05V_CH08

Every pin can be used through the built-in functions of the Arduino programming language.

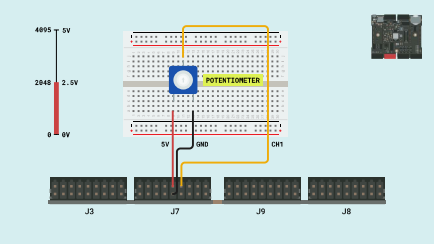
Edge Control ADC can be configured to 8, 10 or 12 bits defining the argument of the following function respectively (default is 10 bits):

```
1  analogReadResolution(12);
```

The Edge Control ADC reference voltage is fixed to 5V, which means the ADC range will be mapped from 0 to 5.0 volts.

The example code below reads the analog input value from every Edge Control channel. It displays it on the IDE Serial Monitor:

This example code can also be found on **File > Examples >**




```
/*
  Testing strategy: conn
*/

#include <Arduino_EdgeC

constexpr unsigned int a

constexpr pin_size_t inp
    INPUT_05V_CH01,
    INPUT_05V_CH02,
    INPUT_05V_CH03,
    INPUT_05V_CH04,
    INPUT_05V_CH05,
    INPUT_05V_CH06,
    INPUT_05V_CH07,
    INPUT_05V_CH08
};
constexpr size_t inputCh
int inputChannelIndex {

struct Voltages {
    float volt3V3;
    float volt5V;
};

void setup()
{
```

IRQ Inputs

The Edge Control has **six interrupt request input pins**, mapped as follows:

Input Name	Arduino Pin Mapping
Interrupt Request Input 1	IRQ_CH1

Input Name	Arduino Pin Mapping
Interrupt Request Input 2	IRQ_CH2
Interrupt Request Input 3	IRQ_CH3
Interrupt Request Input 4	IRQ_CH4
Interrupt Request Input 5	IRQ_CH5
Interrupt Request Input 6	IRQ_CH6

The IRQ inputs of the Edge Control can be used through the built-in functions of the Arduino programming language. The configuration of an interrupt pin is done in the `setup()` function with the function `attachInterrupt()` as shown below:

```
1  attachInterrupt(digitalPin
```

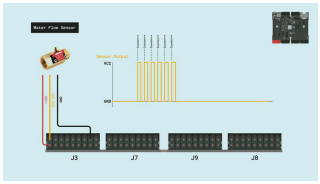
The `pin` argument defines the input channel that will fire the interrupt.

The `ISR` argument defines the interrupt's callback function.

The `mode` argument defines when the interrupt should be triggered.

The example code shown below counts the pulses on every IRQ input and prints the counter value on the IDE Serial Monitor:

This example code can also be found on **File > Examples > Arduino_EdgeControl > Basic > IRQCounter**



IRQ input example wiring

```
/*
    Testing strategy: al
    WAKEUP 1-6 and any c

    Check IRQChannelMap
*/

#include <Arduino_EdgeC

volatile int irqCounts[6

enum IRQChannelsIndex {
    irqChannel1 = 0,
    irqChannel2,
    irqChannel3,
    irqChannel4,
    irqChannel5,
    irqChannel6
};


void setup()
{
    EdgeControl.begin();

    Serial.begin(115200);

    // Wait for Serial M
```

4 - 20 mA Inputs

The Edge Control has **four 4-20mA input pins**, mapped as follows:

Input Name	Arduino Pin Mapping
4 - 20 mA Sensor Input 1	INPUT_420mA_CH01

Input Name	Arduino Pin Mapping
4 - 20 mA Sensor Input 3	INPUT_420mA_CH03
4 - 20 mA Sensor Input 4	INPUT_420mA_CH04

Every 4 - 20 mA input can be read through the built-in functions of the Arduino programming language. They are sampled the same way as the 0 - 5 V analog inputs but the input value is read via a 220 ohm resistor and a +19 V reference.

A current of 4 to 20 mA passing through the 220 ohm resistor would produce a drop of 0.88 V to 4.4 V, respectively, which is in the 5 V range of the ADC input.

A 4 - 20 mA water level sensor will be used in the application example. To convert the analog read voltage back to a current value, the following equation from a 4 - 20 mA sensor can be used:

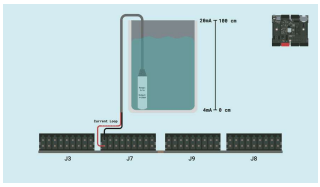
```
y = 16x + 4
```

Where:

- First we solve for x, having the formula: `x = (y - 4)/16` where x is in meters.
- To be able to work in centimeters, you can multiply the expression by 100: `x = (y - 4)*(100/16)`.
- Eventually, you can simplify the expression as: `x = (y - 4)*6.25`.

This is a brief explanation of the mathematical expression used inside the sketch to convert the original sensor value voltage into centimeters:

```
float w_level =
((voltsReference / 220.0 *
1000.0) - 4.0) * 6.25;
```



4 - 20 mA example wiring

```
#include <Arduino_EdgeC

constexpr unsigned int a

struct Voltages {
    float volt3V3;
    float voltRef;
};

void setup()
{
    Serial.begin(115200)

    auto startNow = mil
    while (!Serial && mi
        ;

    delay(1000);
    Serial.println("Hel

    Power.on(PWR_3V3);
    Power.on(PWR_VBAT);
    Power.on(PWR_19V);

    Wire.begin();
    Expander.begin();

    Serial.print("Waitir
```

Watermark Inputs

The Edge Control has **16x Watermark sensor inputs**, mapped as follows:

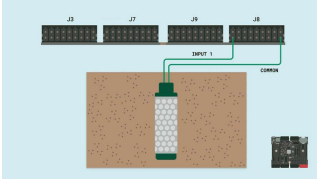
Input Name	Arduino Pin Mapping
Watermark Sensor Input 1	WATERMARK_CH01
Watermark Sensor Input 2	WATERMARK_CH02
Watermark Sensor Input 3	WATERMARK_CH03
Watermark Sensor Input 4	WATERMARK_CH04
Watermark Sensor Input 5	WATERMARK_CH05
Watermark Sensor Input 6	WATERMARK_CH06
Watermark Sensor Input 7	WATERMARK_CH07
Watermark Sensor Input 8	WATERMARK_CH08
Watermark Sensor Input 9	WATERMARK_CH09
Watermark Sensor Input 10	WATERMARK_CH010
Watermark Sensor Input 11	WATERMARK_CH011
Watermark Sensor Input 12	WATERMARK_CH012
Watermark Sensor Input 13	WATERMARK_CH013
Watermark Sensor Input 14	WATERMARK_CH014
Watermark Sensor Input 15	WATERMARK_CH015
Watermark Sensor Input 16	WATERMARK_CH016

Watermark sensors can measure the physical force holding water in the soil. Those measurements are correlated with the effort plants have to make to extract water from the

soil, which is really interesting data for agricultural applications.

The measurement is done in Centibars, and we can use the following readings as a general guideline:

- 0 - 10 Centibars: Saturated soil
- 10 - 30 Centibars: Soil is adequately wet (except coarse sands, which are drying)
- 30 - 60 Centibars: Usual range for irrigation (most soils)
- 60 - 100 Centibars: Usual range for irrigation in heavy clay
- 100 - 200 Centibars: Soil is becoming dangerously dry - Proceed with caution!



Watermark sensor wiring

Watermark sensors are resistive, so the Edge Control measures a resistance value that needs to be converted to a pressure unit, in this case, `centibars` or `kPa`. For this, the function below is used:

```
/**
  This function convert
  @param res is the resi
  @return CB, the centib
*/
int CalcCB(int res) {
  int CB = 0;
  if (res > 550.00) {

    if (res > 8000.00)
      CB = -2.246 - 5.23
    } else if (res > 100
      CB = (-3.213 * (re
    } else {
      CB = ((res / 1000.
    }
  } else {
    if (res > 300.00) {
      CB = 0.00;
    }
    if (res < 300.00 &&
      CB = short_CB;  //
    }
  }

  if (res >= open_resist
    CB = open_CB;  //255
  }
```

The example code below reads the resistance value from the 1st Watermark sensor channel. It displays it on the IDE Serial Monitor in terms of `ohms` and `centibars or kPa`:

More example codes can also be found on **File > Examples > Arduino_EdgeControl > Basic**.

Install the required libraries from their respective URLs using the Arduino Library Manager.

```
#include <Arduino.h>
#include <mbed.h>

#include <Arduino_Edge(
#include <RunningMedia

constexpr unsigned int

mbed::LowPowerTimeout T

uint8_t watermarkChanne

constexpr float tauRati
constexpr float tauRati
constexpr unsigned long

constexpr unsigned int
RunningMedian measures

constexpr unsigned int
RunningMedian calibs {

// Watermark sensors th
const long open_resista


void setup()
{
```

You should see the sensor readings as below, in the Arduino IDE serial monitor.

```
MEASURES - Median: 1891.00Ω -
Average: 1884.95Ω - Lowest:
1815.00Ω - Highest: 1930.00Ω 14
CENTIBARS/kPa
```

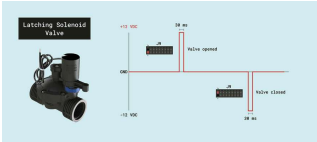
You can buy the Watermark sensor directly from our **store**

Outputs

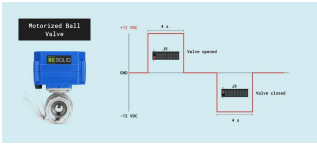
Latching Outputs

The latching outputs are suitable for latching devices like motorized valves and latching solenoid valves. They

can be sent in either of the 2 channels (to open a closed valve for example). The duration of the strobes can be configured to adjust to the external device's requirement.



Latching solenoid valve control strobe



Motorized ball valve control strobe

The board provides a total of 16 latching ports divided into two types:

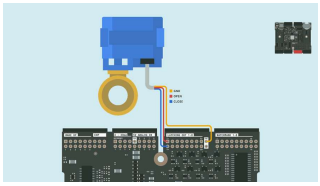
8x Latching Outputs with mosfet drivers, mapped as follows:

Output Name	Arduino Pin Mapping
Latching Output 1	LATCHING_OUT_1
Latching Output 2	LATCHING_OUT_2
Latching Output 3	LATCHING_OUT_3
Latching Output 4	LATCHING_OUT_4
Latching Output 5	LATCHING_OUT_5
Latching Output 6	LATCHING_OUT_6
Latching Output 7	LATCHING_OUT_7

These outputs can handle up to 3.3 A, so they can manage loads directly without problem. Motorized or solenoid latching valves are perfect examples of devices to control with these outputs.

With the following command, you can control the output state:

```
Latching.channelDirection(
Latching.strobe(200);  //t
```



3-wire motorized ball valve connection

To learn more about using these outputs, follow our guide: **Connecting and Controlling a Motorized Ball Valve.**

8x Latching Commands without drivers, mapped as follows:

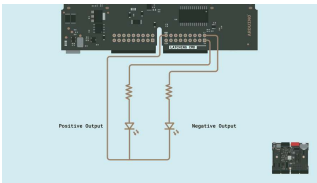
Output Name	Arduino Pin Mapping
Latching Command 1	LATCHING_CMD_1
Latching Command 2	LATCHING_CMD_2
Latching Command 3	LATCHING_CMD_3
Latching Command 4	LATCHING_CMD_4

Output Name	Arduino Pin Mapping
Latching Command 6	LATCHING_CMD_6
Latching Command 7	LATCHING_CMD_7
Latching Command 8	LATCHING_CMD_8

These outputs must be connected to external devices through third-party protection/power circuits with high impedance inputs (max +/- 25 mA). They are suitable for custom applications where just the activation signal is needed, such as external relay modules or direct connections with other control devices like PLC inputs.

With the following command, you can control the output state:

```
1 Latching.channelDirection(
2 Latching.strobe(200);  //t
```



LED pilots wired to latching command outputs

The example code below activates the first channel `Latching Outputs` and `Latching Commands` for a defined time (strobe) in a sequence:

This example code can also be found on **File > Examples >**

```
#include <Arduino_EdgeC

void setup()
{
    Serial.begin(9600);

    auto startNow = mill
    while (!Serial && mi
        ;

    delay(1000);
    Serial.println("Hell

    Latching.begin();
}

void loop()
{
    Latching.channelDire
    Latching.strobe(2000

    Latching.channelDire
    Latching.strobe(2000

    Latching.channelDire
    Latching.strobe(4000

    Latching.channelDire
```

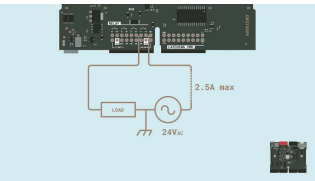
Relay Outputs

The Edge Control has **four solid state relay outputs**, mapped as follows:

Output Name	Arduino Pin Mapping
Solid State Relay 1	RELAY_CH01
Solid State Relay 2	RELAY_CH02
Solid State Relay 3	RELAY_CH03
Solid State Relay 4	RELAY_CH04

These relay outputs are suitable for AC loads with a current draw below 2.5 A and a 24 V AC power supply. You can control the relay outputs individually using this function:

```
Relay.on(RELAY_CH01);  //

Relay.off(RELAY_CH01);  //
```



AC load wiring through channel 1 relay contact

The example code below closes and opens the first channel `Relay Contact` repetitively, turning on and off the connected load:

```
#include "Arduino_EdgeC

// #define SSR_POLL

constexpr unsigned long
constexpr unsigned long
constexpr unsigned long
unsigned long offTime;
unsigned long onTime;
unsigned long pollTime;
bool on = false;

int relayChannel { RELA

void setup()
{
    Serial.begin(9600);
    while (!Serial)
        ;

    delay(2000);

    Serial.println("Hel

    Power.on(PWR_3V3);
    Power.on(PWR_VBAT);

    Wire.begin();
```

Power Rails

The Edge Control lets you manage the power state of several internal components, so energy-efficient applications can be achieved using this board.

To power on or off a power rail, the `Power.on(<rail>)` and `Power.off(<rail>)` functions are used.

The different commands and controllable power rails will be specified in the list below:

```
Power.on(PWR_VBAT); // tur
Power.on(PWR_3V3); // turn
Power.on(PWR_19V);  // tur
Power.on(PWR_MKR1); // tur
Power.on(PWR_MKR2); // tur
```

Every power rail from above can be turned off using the `Power.off(<rail>)` function.

Edge Control Enclosure Kit

Edge Control Enclosure Kit

Designed for industrial and smart agriculture applications, the Arduino Edge Control Enclosure Kit is the perfect companion for Arduino Edge Control. It provides the module with a sturdy case that protects it from the elements, dust, and accidental blows. It is IP40-certified and compatible with DIN rails, making it safe and easy to fit in any standard rack or cabinet.

In addition, the Arduino Edge Control Enclosure Kit features a 2-row/16-character LCD display with a white backlight and a programmable push button. Users can customize it to instantly visualize sensor data, such as weather conditions and soil parameters. Different data can be displayed at every push of the button, on the spot and in real time,

LCD Control

The functions for controlling the LCD are mostly the same as those used in the Arduino ecosystem.

We need to include the `Arduino_EdgeContro.h` library and initialize the LCD with the `LCD.begin(16, 2)` function and start printing data on it.

With `LCD.backlight()` and `LCD.noBacklight()`, we can turn on or off the LCD backlight respectively.

To define the cursor position, `LCD.setCursor(x, y)` is used, and to clear the display, `LCD.clear()`.

We use the `LCD.print(text)` function to display text.

The example code below prints "Edge Control" in the first row and starts a seconds counter in the second one:

```
#include <Arduino_EdgeCon

void setup() {

  // set up the LCD's num
  LCD.begin(16, 2);
  LCD.backlight();
  // Print a message to t
  LCD.home();  // go home
  LCD.print("Edge Control

}

void loop() {

  LCD.setCursor(0, 1);
  LCD.print(millis() / 10
  delay(1000);
}
```

LCD demo running the code from above

Push Button

The Enclosure Kit includes a push button, so you can easily interact with the device.

To read the button state, we can use the built-in functions of the Arduino programming language. First, we need to define it as an input using the `POWER_ON` macro.

```
1  pinMode(POWER_ON, INPUT);
```

The state of the button can be read as usual with the `digitalRead(POWER_ON)` function.

> When the button is pressed, the input state gets low.

In the example code below, we will attach the input to an interrupt and increase a counter with every tap shown in the LCD.

```
#include <Arduino_EdgeC

// Keep track of toggle-
volatile bool buttonPres
bool ledStatus{ false };

void setup() {

  pinMode(POWER_ON, INPU
  // ISR for button pres
  attachInterrupt(
    digitalPinToInterrup
      buttonPressed = tr
    },
    FALLING);

  // set up the LCD's nu
  LCD.begin(16, 2);
  LCD.backlight();
  // Print a message to
  LCD.home();  // go hom
  LCD.print("Push Buttor
}

void loop() {
  static int counter = 0
  if (buttonPressed == t
    buttonPressed = fals
```

LCD and Push Button demo running the code from above

RTC

The built-in Real Time Clock of the Edge Control is ideal for timing irrigation processes and data logging.

To maintain the RTC on time the included CR2032 coin cell must be used.

An example code to test the RTC functionality can be found on **File > Examples > Arduino_EdgeControl > Basic > RealTimeClock**.

In the example code, a `.h` file called Helpers includes useful and easy-to-use functions for parsing the time on different formats.

Initially, you must set the current time as a reference for the RTC. This is made just once and can be done using the function setEpoch(**Current Unix Time**):

```
RealTimeClock.setEpoch(Uni
```

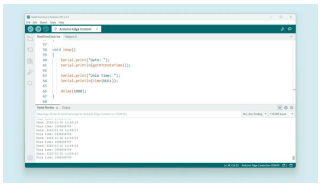
The `time(NULL)` function returns the Unix time (seconds since Jan 1st, 1970), perfect for logging data in the cloud and servers.

```
getRTDateTime(); // 2023-0
getRTCTime(); // 16:33:19
getRTCDate(); // 2023-09-3
```

Also, you can use these more specific functions to retrieve the time in a custom manner:

```
RealTimeClock.getYears();
RealTimeClock.getMonths();
RealTimeClock.getDays();
RealTimeClock.getHours();
RealTimeClock.getMinutes()
RealTimeClock.getSeconds()
```

set the time once with the code build date. (The button should be pressed while the code is uploaded or during a hard reset).



RealTimeClock example output

Communication

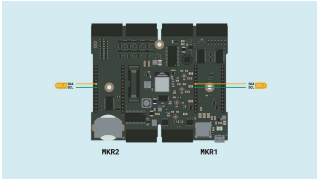
The Edge Control communication peripherals are not accessible to the user as a normal development board because it isn't. For example, the `SPI` communication is reserved for the micro SD card and external memory ICs.

I2C

The pins used in the Edge Control for the I2C communication protocol are on the MKR slots. To find them on the board, refer to the **board pinout section** of the user manual.

The Edge Control supports I2C communication, which allows data transmission between the board and other I2C-compatible devices. Internal components like the LCD driver, the Real-Time Clock, and the I/O expanders use this protocol.

The NINA-B306 has two I2C ports. The `I2C_1` is shared with the internal components and the MKR1 header, and the `I2C_2` is exclusively connected to the MKR2 header.



Edge Control I2C Pins on MKR Connectors

Use `Wire` to communicate with MKR1 (`I2C_1`), and `Wire1` to communicate with MKR2 (`I2C_2`).

To use I2C communication, include the `Wire` library at the top of your sketch. The `Wire` library provides functions for I2C communication:

```
#include <Wire.h>
```

In the setup() function, initialize the I2C library:

```
// Initialize the I2C comm
Wire.begin();   // Wire to
```

To transmit data to an I2C-compatible device, you can use the following commands:

```
// Replace with the targe
byte deviceAddress = 0x05

// Replace with the appro
byte instruction = 0x00;

// Replace with the value
byte value = 0xFF;

// Begin transmission to
Wire.beginTransmission(de

// Send the instruction b
Wire.write(instruction);

// Send the value
Wire.write(value);

// End transmission
Wire.endTransmission();
```

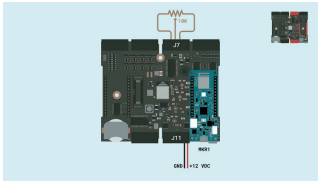
To read data from an I2C-compatible device, you can use the `requestFrom()` function to request data from the device and the `read()` function to read the received bytes:

```
// The target device's I2
byte deviceAddress = 0x05

// The number of bytes to
int numBytes = 2;

// Request data from the
Wire.requestFrom(deviceAd

// Read while there is da
while (Wire.available())
  byte data = Wire.read()
}
```

In the example code below, we will establish communication between the `Edge Control` and an `MKR WiFi 1010` connected to the

Control, the onboard `LED` of the `MKR board` will be controlled, so we will send the brightness value through I2C to it.

> Use the same connection from **this section** for the potentiometer wiring.



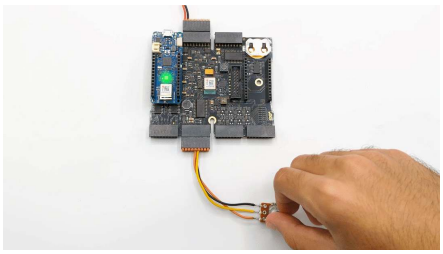
Demo wiring - MKR WiFi 1010 on slot 1 of the Edge Control

Edge Control Code

```
#include <Arduino_EdgeC

// The MKR1 board I2C ac
#define EDGE_I2C_ADDR 0x

constexpr unsigned int a

typedef struct
{

  int LED = 0;  //sharec

} SensorValues_t;

SensorValues_t vals;

void setup() {
  EdgeControl.begin();
  Wire.begin();
  delay(500);
  Serial.begin(115200);
  Serial.println("Init b

  // Enable power lines
  Power.on(PWR_3V3);
  Power.on(PWR_VBAT);
  Power.on(PWR_MKR1);
  delay(5000);
```

MKR WiFi 1010 Code

```
#include <Wire.h>
#include <WiFiNINA.h>
#include <utility/wifi_

// The MKR1 board I2C ac
#define SELF_I2C_ADDR 0x

#define GREEN 26

// I2C communication flc
int ctrlRec = 0;
int ctrlReq = 0;

typedef struct
{

  int LED = 0;  //zone 1

} SensorValues_t;

SensorValues_t vals;

void setup() {
  Serial.begin(115200);
  // put your setup code
  // Init I2C coomunicat
  Wire.begin(SELF_I2C_AD
  Wire.onReceive(receive
```



UART

The pins used in the Edge Control for the UART communication protocol are found on the MKR slots. Refer to the **board pinout section** of the user manual to find them on the board.

To begin with UART communication, you will need to configure it first. In the `setup()` function, set the baud rate (bits per second) for UART communication:

```
// Start UART communicatio
Serial1.begin(115200);
```

To read incoming data, you can use a `while()` loop to continuously check for available data and read individual characters. The code shown above stores the incoming characters in a String variable and processes the data when a line-ending character is received:

```
// Variable for storing i
String incoming = "";
void loop() {
  // Check for available
  while (Serial1.availabl
    // Allow data bufferi
    delay(2);
    char c = Serial1.read

    // Check if the chara
    if (c == '\n') {
      // Process the rece
      processData(incomin
      // Clear the incomi
      incoming = "";
    } else {
      // Add the characte
      incoming += c;
    }
  }
}
```

To transmit data to another device via UART, you can use the `write()` function:

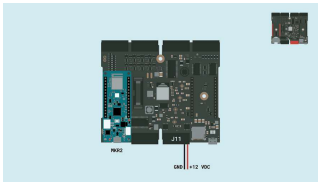
```
// Transmit the string "He
Serial1.write("Hello world
```

You can also use the `print` and `println()` to send a string without a newline character or followed by a newline character:

```
// Transmit the string "He
Serial1.print("Hello world
// Transmit the string "He
Serial1.println("Hello wor
```

To learn more about how to communicate the Edge Control through UART with other devices, we will use the example code below that can be found on **File > Examples > Arduino_EdgeControl > RPC > BlinkOverSerial**.

This example shows how to export Arduino functions as remote procedure calls (RPC), the Edge Control is capable of controlling a MKR board built-in LED by Serial communication.



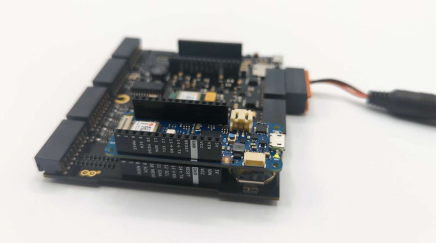
Connections for the UART example

Edge Control Code

```
#include <Arduino_EdgeCor

bool led { false };

void setup()
{
    EdgeControl.begin();
    Power.on(PWR_3V3);
    Power.on(PWR_VBAT);
    Power.on(PWR_MKR2);

    // Wait for MKR to po
    delay(5000);

    SerialMKR2.begin(9600
    while (!SerialMKR2) {
        delay(500);
    }
}

void loop()
{
    SerialMKR2.write(led)
    led = !led;
    delay(1000);
}
```

MKR Code

```
void setup() {
      Serial1.begin(9600)
      pinMode(LED_BUILTIN
      digitalWrite(LED_BU

      delay(1000);
    }

    void loop() {
      if (Serial1.availab
        auto c = Serial1.
        digitalWrite(LED_
      }
    }
```



Micro SD Card

A Micro SD Card can be integrated into the Edge Control for sensor data logging, storing project configurations, or any application concerned with data storage.

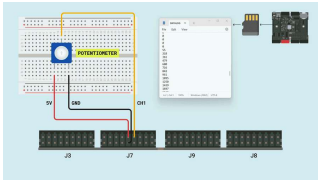
The well-known **SD library** is compatible with Edge Control. We can test the included examples by adding some lines of code that Edge Control needs.

The example code below is an adaptation of the `Datalogger` example in the `SD` library. It logs the value sampled from an analog input of the Edge Control.

The same wiring of the **analog input section** can be used with the attached micro SD card.

```
#include <Arduino_EdgeC
#include <SPI.h>
#include <SD.h>

constexpr unsigned int a

const int chipSelect = F

void setup() {
  // Open serial communi
  Serial.begin(115200);
  while (!Serial) {
    ;  // wait for seria
  }

  EdgeControl.begin();
  // Power on the 3V3 ra
  Power.on(PWR_3V3);
  Power.on(PWR_VBAT);

  Wire.begin();
  Expander.begin();

  Serial.print("Waiting
  while (!Expander) {
    Serial.print(".");
    delay(100);
  }
```

You can read all the sampled data from the micro SD card on a `.txt` file called `DATALOG` using a USB adapter or a micro SD card slot on your PC.



Data logging wiring and output

Support

If you encounter any issues or have questions while working with the

support resources to help you find answers and solutions.

Help Center

Explore our **Help Center**, which offers a comprehensive collection of articles and guides for the Edge Control. The Arduino Help Center is designed to provide in-depth technical assistance and help you make the most of your device.

Forum

Join our community forum to connect with other Edge Control users, share your experiences, and ask questions. The forum is an excellent place to learn from others, discuss issues, and discover new ideas and projects related to the Edge Control.

> **Edge Control category in the Arduino Forum**

Contact Us

Please get in touch with our support team if you need personalized assistance or have questions not covered by the help and support resources described before. We're happy to help you with any issues or inquiries about the Edge Control.

> **Contact us page**

public GitHub repository. If you see anything wrong, you can edit this page here.

Share Alike 4.0 license.

Was this article helpful?